

How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkhanov

*Bachelor Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

seylkhanov.aldiyar@gmail.com

Alexandr Tyulkov

*Master Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

widesehl@gmail.com

0009-0009-6422-0559

Abstract—TypeScript monorepos often combine frontend, backend, and shared packages under one repository, but test execution is still shaped by package-local configuration. This paper studies how a unified toolchain affects testing consistency in such a setting. The Circles repository is used as a case study because it runs backend, frontend, and shared-package tests through the Vite+ command interface while still keeping a small number of package-specific overrides. The evaluation uses repository inspection, CI workflow analysis, and direct execution of test and coverage commands. The observed configuration surface consists of nine testing and CI configuration files, seven automated test files, and two GitHub Actions workflows. Empirical results show stable execution in the backend and shared package, with 21 backend tests and one shared-package test passing, while the aggregated frontend run exposes a configuration-level failure that does not appear when the frontend unit project is executed in isolation. This result is important because it shows that a unified command surface improves consistency, but multi-project composition can still preserve hidden environment differences. The paper therefore argues that unified toolchains reduce configuration fragmentation and improve reproducibility, yet consistency must still be evaluated at the combined pipeline level rather than only at the per-project level.

Index Terms—monorepo, TypeScript, unified toolchain, automated testing, Vite, software engineering

I. INTRODUCTION

TypeScript monorepos put frontend, backend, and shared packages in one repository. This makes code sharing easier. But it also makes the toolchain more complex. Different packages can end up using different test runners, module resolvers, and build transforms. When that happens, the same test logic can pass in one package and fail in another. The code is not wrong. The environment is different.

This problem is common in full-stack repositories. Packages are not truly isolated. They share types, utility modules, and build assumptions. When each package has its own testing setup, small differences in configuration can produce different results. This is called toolchain fragmentation. It is an environment problem, not a logic problem.

This paper studies that problem through Circles, a TypeScript full-stack monorepo. Circles uses a unified toolchain

called Vite+ that gives all packages one command interface and one core testing stack. The study examines whether this unification produces consistent test behavior across packages. It combines repository inspection, CI workflow review, and direct test execution. Three research questions guide the analysis:

- RQ1** How does per-package toolchain fragmentation create inconsistent test behavior in a TypeScript monorepo?
- RQ2** Does a unified toolchain reduce the number of configuration files needed to run tests across packages?
- RQ3** Does sharing one command interface and one transform pipeline produce more uniform test results in Circles?

The paper makes three contributions. First, it defines toolchain fragmentation as an environment-level cause of test inconsistency. Second, it documents the Circles testing setup using observable repository evidence. Third, it evaluates unification using configuration surface size, execution outcomes, coverage scope, and CI pipeline structure.

II. RELATED WORK

Related work falls into two areas. The first area covers software quality and fault patterns in TypeScript projects. The second area covers automated testing in continuous integration environments.

A. TypeScript Quality and Fault Profiles

Bogner and Merkel compared 604 JavaScript and TypeScript projects on GitHub [1]. They measured code quality, understandability, bug count, and bug resolution time. TypeScript projects scored better on quality and understandability. But they did not show significantly fewer bugs or faster fixes. This is relevant because it limits a common assumption. Using TypeScript improves some code properties, but it does not remove the conditions that cause bugs during testing.

Tang, Alimadadi, and Sumner analyzed 633 bug reports from 16 TypeScript repositories [2]. They built a taxonomy of fault categories. The main finding is that tooling and configuration faults are more common than logic or syntax errors. Many failures appear at integration and orchestration

boundaries. This directly supports the premise of this paper. In TypeScript projects, fragility often lives in the surrounding toolchain rather than in the application code.

Both studies are useful but stop short of the research question here. Bogner and Merkel compare repositories at a population level. Tang et al. describe fault patterns across the ecosystem. Neither paper asks whether sharing one resolver and one test runner across packages changes consistency inside a single monorepo.

B. Continuous Integration and Automated Testing

Wang et al. studied test automation maturity in CI environments [3]. Higher maturity correlated with better product quality and shorter release cycles. Reliable test infrastructure matters. But the unit of analysis is the project as a whole, not differences between packages inside one repository.

Yu et al. examined automated non-functional testing in CI through an industry study [4]. They describe CI as a system of tools, metrics, and feedback mechanisms. Test behavior depends on the surrounding infrastructure, not only on the test cases. This supports the framing of the current paper. But that study focuses on NFR evaluation, not on per-package differences in test stack configuration.

The reviewed literature shows three things. TypeScript improves code quality indicators but does not eliminate bug-related problems [1]. Failures in TypeScript projects often cluster around configuration and tooling boundaries [2]. Testing outcomes depend on automation infrastructure [3]; [4]. What is missing is a study of how toolchain unification affects test consistency across packages in a single monorepo. That gap motivates this case study.

III. METHODOLOGY

A. Case Study Context

Circles is a full-stack TypeScript monorepo. It has three active package areas relevant to testing. The frontend uses React and contains browser component tests and end-to-end tests. The backend exposes API and worker code and uses unit tests for logic-level checks. The shared utilities package contains reusable helpers with its own test file. All packages use Vite+ as the main toolchain, accessed through a single CLI.

The repository provides a useful case because it combines a unified command interface with some package-level specialization. The backend and shared package each define testing inside their main Vite config file. The frontend keeps a base config plus dedicated files for unit testing, browser testing, and end-to-end testing. This structure allows direct observation of where unification works and where configuration differences remain.

TABLE I
PACKAGE STRUCTURE AND TESTING STACK IN CIRCLES

Package	Primary role	Test stack	Test files
Frontend	UI and route rendering	Vitest unit, Vitest browser, Playwright	3
Backend	API, workers, helper logic	Vitest through shared Vite config	4
Shared utils	Reusable helper package	Vitest through shared Vite config	1

B. Data Collection

The evaluation used only repository-observable evidence. Five types of data were collected.

First, the repository structure was inspected to find configuration files, package scripts, and test file locations. Second, the CI workflows were reviewed to document the execution model used on push and pull request events. Third, direct test execution was performed using the commands defined in the repository for each package. Fourth, execution logs were examined for failures, warnings, and output patterns. Fifth, coverage reports were used to identify the current automated scope.

All observations were made against the repository state on 2026-04-10.

C. Evaluation Dimensions

The analysis used four dimensions.

- 1) **Configuration surface size.** The number of files that directly define test or CI behavior.
- 2) **Execution consistency.** Whether equivalent test commands produce the same outcome across packages and across isolated versus aggregated runs.
- 3) **Coverage and detectability.** Which modules have automated test coverage and how much of the system is visible to the test suite.
- 4) **Pipeline reproducibility.** How repository test commands map into CI workflows.

TABLE II
EVALUATION DIMENSIONS AND EVIDENCE SOURCES

Dimension	Metric	Evidence source
Configuration surface	Count of config files	Vite, Vitest, Playwright, and GitHub Actions files
Execution consistency	Pass or fail outcome	Direct test runs across backend, frontend, and shared package
Coverage and detectability	Coverage percentage	V8 coverage reports from package coverage commands
Pipeline reproducibility	Step mapping	CI workflow YAML files and package scripts

D. Configuration and Pipeline Model

Nine files form the testing and CI configuration surface. Two are GitHub Actions workflows. Three are Vite config files, one per package. Three are frontend-specific Vitest

config files. One is the Playwright config for end-to-end tests. This count is small for a multi-package repository but it is not zero. Unification in Circles is real but not absolute.

TABLE III
CI PIPELINE STRUCTURE ACROSS THE TWO WORKFLOWS

Push / PR	→	Install	→	Check and test	→	Build or E2E
GitHub event	→	Install dependencies	→	Static checks and recursive test run	→	Recursive build or Playwright E2E

The main CI workflow performs checkout, dependency installation, static analysis, recursive test execution, and recursive build. A second workflow handles end-to-end tests using a browser runner. Both workflows use the same unified command interface. The frontend still requires a separate E2E workflow, which shows that some package-specific steps remain even under a unified toolchain.

IV. PRELIMINARY RESULTS

A. Configuration Surface and Test Inventory

Repository inspection found seven automated test files and two CI workflows. Of the seven test files, six belong to unit or component-style automation and one belongs to end-to-end testing. The backend contributes four spec files, the frontend contributes one unit file, one browser component file, and one Playwright file, and the shared package contributes one test file. This distribution shows that the current automated baseline is concentrated on helper logic and lightweight UI checks rather than on cross-package interaction paths.

TABLE IV
OBSERVED TESTING INVENTORY AND CONFIGURATION SURFACE

Area	Unit / component	E2E	Config files	Main config locations
Frontend	2	1	5	Vite config, three Vitest configs (aggregated, unit, browser), Playwright config
Backend	4	0	1	Vite config
Shared utils	1	0	1	Vite config
CI workflows	N/A	N/A	2	Unit CI workflow, E2E CI workflow

The frontend has the richest test-stack composition and the largest local configuration surface. The backend and shared package each depend on a single Vite+ configuration file. This asymmetry is relevant to the later failure analysis because the only observed inconsistency during direct execution appears in the frontend package.

B. Direct Execution Outcomes

Direct test execution produced stable results in the backend and shared package. The backend test suite completed successfully with 4 passing files and 21 passing tests. The shared utilities suite completed successfully with 1 passing file and 1 passing test. The frontend produced mixed results. The aggregated frontend test command reported one passing file, one failed suite, and a runner initialization failure in the unit test suite. However, running the unit suite in isolation through its dedicated configuration passed with 1 passing file and 4 passing tests.

TABLE V
OBSERVED EXECUTION OUTCOMES ON 2026-04-10

Run scope	Package	Files	Tests	Result	Notes
Backend test suite	Backend	4 passed	21 passed	Pass	Stable package-level execution
Frontend aggregated	Frontend	1 passed, 1 failed	2 passed	Fail	Initialization error in unit suite during multi-project run
Frontend unit (isolated)	Frontend unit only	1 passed	4 passed	Pass	Unit project succeeds in isolation
Shared utils suite	Shared utils	1 passed	1 passed	Pass	Stable single-package execution

The failing frontend run is important because it changes the interpretation of consistency. A shared CLI entry point is present, but consistent outcomes are not guaranteed when multiple frontend test projects are composed under one aggregate command. The initialization error appears before unit assertions execute, indicating an environment or runner composition problem rather than a fault in the tested utility function itself.

C. Coverage and Detectability

Coverage commands show strong detectability for the currently targeted helper modules and very limited breadth beyond that scope. Backend coverage reached 100% statement, branch, function, and line coverage across four helper modules. Frontend unit coverage reached 100% for the UI utility module. Shared-package coverage reached 100% for the package entry point. These results confirm that the automated baseline is precise but narrow.

TABLE VI
OBSERVED COVERAGE FOR THE CURRENTLY INSTRUMENTED SCOPE

Package	Covered target	State-ments	Branches	Func-tions	Lines
Backend	Four helper utility modules	100%	100%	100%	100%
Frontend unit	UI utility module	100%	100%	100%	100%
Shared utils	Pack-age entry point	100%	100%	100%	100%

The coverage data should not be interpreted as system-wide completeness. In the frontend, coverage is restricted to one utility module. In the backend, controller and workflow layers remain outside the reported coverage target set even though they are likely to carry higher integration risk. Therefore, detectability is high inside the instrumented helper scope and lower outside it.

D. Failure Pattern Analysis

Only one concrete failure was observed during direct execution, but it is analytically useful because it was not predicted by simple per-file unit reasoning. The failure affected the frontend unit suite during the aggregated frontend command and did not reproduce in the isolated unit-project run. This creates a difference between local project success and combined pipeline behavior.

TABLE VII
OBSERVED FAILURE EVIDENCE

Failure ID	Affected module	Failure type	Frequency	Interpretation
F01	Frontend unit test suite	Runner initialization error during aggregated run	1 observed aggregated run	The command surface is unified, but the composed frontend test environment is still sensitive to project interaction

This result increases the apparent risk of frontend test orchestration and lowers confidence in detectability at the package boundary. The backend and shared package currently show no analogous evidence of instability. At this stage, the strongest preliminary finding is therefore mixed: unification simplifies the command surface and supports reproducible coverage reporting, but consistency claims remain weaker in the most configuration-heavy package.

V. DISCUSSION

The results support a limited but clear claim. Circles shows that toolchain unification improves reproducibility at the command and CI level. Most test commands go through one interface. Coverage output looks similar across packages.

The CI workflows are compact and follow a shared structure. These are real benefits.

But the frontend failure shows that unification at the command level does not guarantee uniform behavior at the execution level. When the frontend unit and browser projects run together under one aggregated command, a runner initialization error appears. The same unit project passes when run in isolation. This means the environment is sensitive to project composition, even inside a unified stack.

This distinction matters for quality assurance. There are two different things that can be called consistency. The first is command-level consistency: all packages use the same tool and the same interface. The second is execution-level consistency: combined runs produce the same result as isolated runs. Circles currently has the first. The frontend failure shows that the second is not yet guaranteed.

A. Implications for Risk and Test Strategy

The backend helper layer has high detectability and stable execution. Four spec files pass reliably and coverage is complete within the configured scope. The shared package shows the same pattern on a smaller scale.

The frontend should be assessed differently. Its unit scope is small and well-covered. But the aggregated execution failure raises a real risk. A test can pass in isolation and still fail when run as part of a larger pipeline. Isolated success is not enough evidence for release confidence in the frontend package.

The coverage results also need careful interpretation. All observed coverage values are 100%, but they cover only a small part of the system. Controller logic, route-level code, worker orchestration, and end-to-end flows are not covered. Full coverage of a small target scope does not mean the system is well-tested.

B. Threats to Validity

Four threats apply to this study. First, it is a single-repository case study. The findings cannot be generalized without further evidence. Second, there is no before-and-after migration data from the same repository. The paper evaluates the current unified setup and its remaining inconsistencies, but it cannot measure the exact improvement from a previous fragmented state. Third, coverage results reflect the configured target scope, not the full system. Fourth, the frontend failure was observed once during direct execution. It is real evidence of inconsistency, but it is not a flakiness estimate from repeated runs.

C. Generalizability

The case supports a general principle that goes beyond this specific toolchain. When packages share one resolver, one transform pipeline, and one command interface, the number of environment differences that can cause test disagreements is reduced. This principle applies to other unified toolchains as well.

But the Circles case also shows a limit of unification. Browser testing and end-to-end testing have different requirements than unit testing. These differences reintroduce

local configuration even inside a unified setup. A unified toolchain reduces fragmentation. It does not remove the need to test cross-project composition explicitly.

VI. CONCLUSION

This paper examined testing consistency in the Circles TypeScript monorepo. The study used repository inspection, CI workflow analysis, and direct test execution as evidence sources. The main question was whether a unified toolchain produces consistent test behavior across packages.

The findings are mixed but informative. Backend and shared-package test execution was stable. All configured coverage targets reached 100%. The CI workflows are compact and follow a shared structure. These results show that unification reduces the configuration surface and supports reproducible coverage reporting.

The frontend package produced a different result. The aggregated test command failed with a runner initialization error. The same unit project passed in isolation. This failure shows that unification improves conditions for consistency but does not guarantee it. A unified command interface is not the same as a uniform execution environment. Combined runs can still fail in ways that isolated runs do not reveal.

The practical implication is straightforward. Teams using unified toolchains should not treat isolated test success as sufficient evidence for pipeline stability. Aggregated and composed execution must be tested separately. Coverage numbers must be interpreted against scope, not just percentage.

Future work should expand test coverage to controller, route, and workflow layers. Repeated execution data would allow flakiness estimates. A comparison with an older fragmented version of the same repository would strengthen the causal claim about unification benefits.

ACKNOWLEDGMENT

REFERENCES

- [1] J. Bogner and M. Merkel, “To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, ACM, 2022, pp. 658–669. doi: 10.1145/3524842.3528454.
- [2] T. Tang, S. Alimadadi, and N. Sumner, “From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem,” in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2026, pp. 1–11. doi: 10.1145/3793302.3793379.
- [3] Y. Wang, M. V. Mäntylä, Z. Liu, and J. Markkula, “Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration,” *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.
- [4] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, “Automated NFR Testing in Continuous Integration Environments: A Multi-Case Study of Nordic Companies,” *Empirical Software Engineering*, vol. 28, no. 144, 2023, doi: 10.1007/s10664-023-10356-1.